

TIP DÖNÜŞÜMLERİ VE GÖSTERİCİLER

Tip Dönüşümleri

C# dilinde derleme öncesi bir değişken bildirildikten sonra, bu tip örtük olarak bir başka değişkenin tipine dönüştürülebilir olmadığı sürece tekrar bildirilemez veya başka bir tipte bir değer atanamaz. Örneğin, `string` örtük olarak `int` veri tipine dönüştürülemez. Bu nedenle, bir değişken `int` olarak bildirdikten sonra, aşağıdaki koddaki gibi, ona "Merhaba" dizgisini atayamazsınız;

```
int i;
i = "Merhaba"; // error CS0029: can't implicitly convert type 'string' to 'int'
```

Ancak bazen bir değeri başka bir tipin değişkenine veya yöntem parametresine kopyalamanız gerekebilir. Örneğin, `double` parametresi bekleyen bir yöntemle geçirmeniz gereken bir tam sayı değişkeniniz olabilir. Bir başka örnek olarak bir `sınıf örneğini` (`instance`) bir `ara yüz` (`interface`) tipinin değişkenine atamanız gerekebilir. Bu tür işlemlere tip dönüşümleri denir;

- **Örtük dönüşümler** (`implicit conversion`): Dönüştürme her zaman başarılı olduğundan ve veri kaybı olmadığından özel bir söz dizimi gerekmez. Örnek olarak küçükten daha büyük tam sayı türlerine dönüştürmeler ve türetilmiş sınıflardan temel sınıflara dönüştürmeler verilebilir.
- **Açık dönüşümler** (`explicit conversion`) veya **zorlama** (`cast`): Açık dönüşümler bir atama ifadesi gerektirir. Dönüşümde bilgi kaybolabileceği veya dönüştürmenin başka nedenlerle başarılı olmayabileceği durumlarda atama gereklidir. Buna örnek olarak da daha az duyarlığa veya daha küçük bir belleğe sahip bir tipe sayısal dönüştürme ve temel sınıf örneğinin türetilmiş bir sınıfa dönüştürülmesi verilebilir.
- **Kullanıcı tanımlı dönüşümler**: Kullanıcı tanımlı dönüştürmeler, temel sınıf türetilmiş sınıf ilişkisi olmayan özel türler arasında açık ve örtük dönüştürmeleri etkinleştirmek için tanımlayabileceğiniz özel yöntemler kullanır.
- **Yardımcı sınıflarla dönüşümler**: Uyumlu olmayan tipler arasında dönüştürme yapmak için yardımcı sınıflar kullanılır. `System.Convert`, `Parse`, `Int32.Parse` ve `System.BitConverter` bu yardımcı sınıf ve yöntemleridir.

Örtük Dönüşümler

Yerleşik sayısal tipler için, depolanacak değer kırılmadan veya yuvarlanmadan değişkene sığabildiğinde **örtük** (`implicit`) dönüştürme yapılabilir. Tam sayı tipleri için bu, kaynak tipi aralığının hedef tipi için aralığın uygun bir alt kümesi olduğu anlamına gelir. Örneğin, `long` (64 bit tamsayı) türünde bir değişken, `int` (32 bit tamsayı) tarafından depolanabilir herhangi bir değeri depolayabilir.

Aşağıdaki örnekte, derleyici `uzunTamsayı` değişkenine `tamsayı` değerini atamadan önce örtük olarak `long` tipine dönüştürür. Tüm örtük sayısal dönüştürmelerin tam listesi için dipnottaki linke bakabilirsiniz²².

```
int tamsayı = 2147483647;
long uzunTamsayı = tamsayı;
```

Referans tipler için, örtük dönüştürme her zaman bir sınıftan doğrudan veya dolaylı temel sınıflarından veya arabirimlerinden herhangi birine var olur. Türetilmiş bir sınıf her zaman bir temel sınıfın tüm üyelerini içerdiğinden özel söz dizimi gerekmez.

```
Derived d = new Derived();
Base b = d; // Türetilmiş sınıf, her zaman taban sınıfa örtük olarak dönüştürülür.
```

²² <https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/numeric-conversions>

Açık Dönüşümler

Bilgi kaybı riski olmadan dönüştürme yapılamazsa, derleyiciye açık bir dönüştürme gerçekleştirmenizi bildirmeniz gerektirir. Açık (**explicit**) dönüşümler, Derleyiciye dönüştürmeyi amaçladığınızı ve veri kaybı olabileceğini veya atamanın çalışma zamanında başarısız olabileceğini fark ettiğinizi açıkça bildirmenin bir yoludur. Bir atama gerçekleştirmek için, dönüştürülecek değerin veya değişkenin önündeki parantez içinde atama yaptığınız veri tipi belirtilir;

```
double x = 1234.7;
int a;
a = (int) x; // double tipini int tipine dönüştürme için zorlanıyor (cast).
Console.WriteLine(a); //1234 Ondalık kısmı göz ardı edildi!
```

Bu program, **zorlama** (**cast**) olmadan derlenmez. Referans tipler için, bir taban tipten türetilmiş tipe dönüştürmeniz gerekiyorsa zorlama yine gereklidir;

```
Derived d = new Derived();// Türetilmiş tipten bir nesne imal ediliyor.
Base b = d; // Örtük (implicit) olarak tanban tipe dönüştürülebilir.

/*
Ancak bu taban tipteki b nesnesi türetilmiş tipe dönüştürülecek ise açık olarak dönüştürülmesi
gerekir.
*/
Derived d2 = (Derived)b;
```

Referans tipleri arasındaki dönüştürme işlemi, temel alınan nesnenin **çalışma zamanı** (**run time**) tipini değiştirmez. Yalnızca bu nesneye referans olarak kullanılan değerin tipini değiştirir. Yani **çok biçimlilik** (**polymorphism**) burada uygulanır.

Bazı referans tipi dönüştürmelerinde, derleyici bir zorlamanın geçerli olup olmadığını belirleyemez. Doğru derlenen bir açık dönüşüm işleminin çalışma zamanında başarısız olması mümkündür;

```
// See https://aka.ms/new-console-template for more information
Canlı canlı = new Hayvan();
Bitki bitki = (Bitki)canlı; // Derlenir ama çalıştırma anında hata olur:InvalidCastException

class Canlı
{
    public virtual string ToString() => "Ben Bir Canlıyım.";
}
class Hayvan : Canlı {
    public void HareketEt() => System.Console.WriteLine("Hareket Ediyorum..");
}
class Bitki : Canlı {
    public void FotosentezYap() => System.Console.WriteLine("Fotosentez Yapıyorum..");
}
```

Kullanıcı Tanımlı Dönüşümler

Kullanıcı tanımlı bir veri tipi, aynı iki tip arasında standart bir dönüşüm olmadığı sürece, başka bir tipe özel olarak **örtük** (**implicit**) veya **açık** (**explicit**) dönüşüm tanımlayabilir. Örtük dönüşümler, çağırılması için özel bir sözdizimi gerektirmez. Bu dönüşümler atamalarda ve yöntem çağrılarında meydana gelebilir. Önceden tanımlanmış örtük dönüşümleri her zaman başarılı olur ve asla bir istisna oluşturmaz. Kullanıcı tanımlı örtük dönüşümler de bu şekilde davranmalıdır.

Özel bir dönüşüm bir istisna oluşturabilir veya bilgi kaybedebilirse, bunu açık bir dönüşüm olarak tanımlamamız gerekir.

is ve **as** işlemleri kullanıcı tanımlı dönüşümleri dikkate almaz. Kullanıcı tanımlı açık bir dönüşümü çağırmaq için bir dönüştürme ifadesi kullanılması gerekir.

Sırasıyla örtük veya açık bir dönüşümü tanımlamak için **implicit operator** veya **explicit operator** anahtar sözcükleri kullanılır. Bir dönüşümü tanımlayan tip, o dönüşümün kaynak tipi veya hedef tipi olmalıdır. İki kullanıcı tanımlı tip arasındaki bir dönüşüm, iki tipten herhangi birinde tanımlanabilir.

```
// See https://aka.ms/new-console-template for more information
var d = new OndalıkRakam(8);
byte sayı = d;
Console.WriteLine(sayı); // 8
OndalıkRakam rakam = (OndalıkRakam) sayı;
Console.WriteLine(rakam); // 8

public readonly struct OndalıkRakam
{
    private readonly byte rakam;
    public OndalıkRakam(byte b)
    {
        if (b > 9)
        {
            throw new ArgumentOutOfRangeException(nameof(b),
                "Ondalık Rakam 9 dan büyük olamaz..");
        }
        this.rakam = b;
    }
    public static implicit operator byte(OndalıkRakam d) => d.rakam;
    public static explicit operator OndalıkRakam(byte b) => new OndalıkRakam(b);
    public override string ToString() => $"{rakam}";
}
```

Yardımcı Sınıflarla Dönüşümler

Çoğu programımızda klavyeden girilen metni bir **dizgi** (**string**) veri tipinde değişkenlerle işleriz. Bazı durumlarda da bu dizgileri değer tiplere dönüştürürüz. Bunun için **System.Convert** sınıfını kullanırız²³.

```
// See https://aka.ms/new-console-template for more information
using System.Globalization;

DateTime zaman = new(1971, 9, 1, 11, 53, 0);
Console.WriteLine(zaman.ToString("yyyy-MM-dd HH:mm")); // 1971-09-01 11:53
Console.WriteLine(zaman.ToString(CultureInfo.InvariantCulture)); // 09/01/1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("en-us"))); // 9/1/1971 11:53:00 AM
Console.WriteLine(zaman.ToString(new CultureInfo("tr-TR"))); // 1.09.1971 11:53:00
Console.WriteLine(zaman.ToString(new CultureInfo("ja-JP"))); // 1971/09/ 01 11:53:00

string doubleMetin = "12,53";
double sayi = Convert.ToDouble(doubleMetin);
Console.WriteLine(sayi); // 12,53
string intMetin = "6161";
int num = int.Parse(intMetin);
Console.WriteLine(num); // 6161
if (int.TryParse(intMetin, out int result))
{
    Console.WriteLine(result); // 6161
}
else
{
    Console.WriteLine("Dönüştürme Başarısız Oldu.");
}
```

²³ <https://learn.microsoft.com/en-us/dotnet/api/system.convert?view=net-9.0>

Kutulama

Kutulama (**boxing**), bir değer tipini bir nesneye veya bu değer tipi tarafından uygulanan herhangi bir **ara yüz** (**interface**) tipine dönüştürme işlemidir.

```
int i = 123;
object o = i; // o nesnesi i değeri tipini kutulamıştır.
o = 123;
i = (int)o; // i değişkeni kutudan çıkarılmıştır.
object obj = 61;
if (obj is int number)
{
    Console.WriteLine("obj bir tamsayıdır:" + number);
}
```

Göstericiler

C# göstericileri sınırlı bir ölçüde destekler. **Gösterici** (**pointer**), bir bellek adresini tutan bir değişkenden başka bir şey değildir. C# göstericisi yalnızca değer tiplerin ve dizilerin bellek adresini tutmak için bildirilebilir (**declaration**).

1990 yıllarının sonlarına kadar, geliştirme çalışmalarımın çoğu C programlama dilini kullanarak yapıldı. C veya C++ dillerini kullanan programcılar göstericileri kullanmaktan kaçınmazdı. C# ve Java gibi saf nesne yönelimli dilleri kullanmaya başladığımızdan beri göstericiler çok nadir kullanılmaya başlandı ve genellikle buna gerek duyulmamaktadır. Çünkü göstericilerin çokça kullanıldığı dinamik veri yapılarını sağlayan kütüphaneler doğrudan bize altyapı tarafından sunulmuştur. Ancak göstericiler, bazı durumlarda düşük düzey iş ve işlemler için gerekli olabilir. Buna örnek olarak P/Invoke verilebilir. P/Invoke, **yönetilen** (**managed**) kodunuzdan **yönetilmeyen** (**unmanaged**) kütüphanelerdeki yapıları, **geri aramalara** (**callback**) ve fonksiyonlara erişmenizi sağlayan bir teknolojidir²⁴.

Referans tiplerin aksine, gösterici tipleri varsayılan **çöp toplama** (**garbage collection**) mekanizması tarafından izlenmez. Aynı nedenden dolayı göstericilerin bir referans tipe veya hatta bir referans tipi içeren bir **yapı** (**struct**) türüne işaret etmesine izin verilmez. Göstericilerin yalnızca **yönetilmeyen** (**unmanaged**) tipleri, yani tüm temel veri tiplerini (**sbyte**, **byte**, **short**, **ushort**, **int**, **uint**, **long**, **ulong**, **char**, **float**, **double**, **decimal**, **bool**), **enum** tiplerini, diğer gösterici tiplerini ve yalnızca yönetilmeyen tipleri içeren yapıları (**struct**) işaret edebileceğini söyleyebiliriz.

Gösterici Tipi Bildirme

Bir gösterici tipi bildiriminin genel biçimi aşağıda gösterildiği gibidir:

```
veritipi *gösterici-kimliği;
```

Yıldız (*) karakteri, **referanstan çıkarma işleci** (**de-reference operator**) olarak bilinir. Bir **int** tipinin adresini tutabilen bir gösterici değişkeni **ptrToInt** aşağıdaki gibi bildirir;

```
int* ptrToInt;
```

Referans işleci (&) bir değişkenin bellek adresini almak için kullanılabilir. Bir göstericiye ilke değer vermek için kullanılabilir. Aşağıdaki örnekte gösterici veri tipiyle, değişkenin veri tipinin aynı olduğuna dikkat edilmelidir.

```
int x = 100;
int* ptrToInt = & x;
Console.WriteLine((int) ptrToInt) // Konsola ptrToInt göstericisinin tuttuğu adres yazılır
Console.WriteLine(*ptr) // Konsola ptrToInt göstericisinin gösterdiği adresteki değer yazılır.
```

C# ifadeleri **güvenli** (**safe**) veya **güvenli olmayan** (**unsafe**) bir bağlamda yürütülebilir. **unsafe** anahtar sözcüğü kullanılarak güvenli olmayan olarak işaretlenen ifadeler **çöp toplayıcısının** (**garbage**

²⁴ <https://learn.microsoft.com/en-us/dotnet/standard/native-interop/pinvoke>

collector) denetimi dışında çalışır. C# dilinde göstericiler içeren herhangi bir kodun güvenli olmayan bir bağlam gerektirdiğini unutulmamalıdır. **Main** dahil tüm yöntemleri güvenli olmayan olarak işaretleyebilir kullanabiliriz.

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    int x = 1;
    int y = 2;
    int* ptr1 = &x;
    int* ptr2 = &y;
    Console.WriteLine((int)ptr1); //871885156
    Console.WriteLine((int)ptr2); //871885152
    Console.WriteLine(*ptr1); //1
    Console.WriteLine(*ptr2); //2
}
Sınıf nesne = new Sınıf();
nesne.GüvenliOmlayanYöntem();
class Sınıf
{
    public unsafe void GüvenliOmlayanYöntem()
    {
        int x = 3;
        int y = 4;
        int* ptr1 = &x;
        int* ptr2 = &y;
        Console.WriteLine((int)ptr1); //871885052
        Console.WriteLine((int)ptr2); //871885048
        Console.WriteLine(*ptr1); //3
        Console.WriteLine(*ptr2); //4
    }
}
```

C# **çöp toplayıcısı** (**garbage collector**), çöp toplama işlemi sırasında nesneleri istediği gibi bellekte taşıyabilir. Çöp toplayıcısına bir nesneyi taşımamasını söylemek için özel bir **fixed** anahtar sözcüğü kullanılır. Böylece işaret edilen değer türlerinin konumunu bellekte sabitlenir. Bu durum **sabitleme** (**pinning**) olarak bilinir. Aşağıda C dili göstericileri gibi bir dizi elemanlarına erişim ve nesne durumuna erişim örneği verilmiştir;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    var buffer = new int[1024];
    fixed (int* p = &buffer[0])
    {
        for (var i = 0; i < buffer.Length; i++)
        {
            *(p + i) = i;
        }
    }

    Sınıf nesne = new Sınıf();
    nesne.alan = 32;
    nesne.KonsolaAlanıYaz();
    fixed (int* alan = &nesne.alan)
    {
        *alan += 3;
    }
    nesne.KonsolaAlanıYaz();
}

class Sınıf
```

```
{
    public Sınıf() { }
    public int alan; //field
    public void KonsolaAlanıYaz()
    {
        Console.WriteLine("alan = {0}", alan);
    }
}
```

Gösterici Aritmetiği

Göstericilerde toplama ve çıkarma tamsayılardan farklı şekilde çalışır. Bir gösterici artırıldığında veya azaltıldığında, işaret ettiği adres, referans tipinin bellek boyutu kadar artırılır veya azaltılır.

Örneğin, `System.Int32` için takma ad olan `int` tipi 4 bayt boyutundadır. Bir `int`, 0 adresinde saklanabiliyorsa, sonraki `int` 4 adresinde saklanır;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    int* ptr = (int*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr)); // 0
    ptr++;
    Console.WriteLine(new IntPtr(ptr)); // 4
    ptr++;
    Console.WriteLine(new IntPtr(ptr)); // 8
}
```

Benzer şekilde, `System.Int64` için takma ad olan `long` tipi 8 bayt boyutundadır. Eğer bir `long` 0 adresinde saklanabiliyorsa, sonraki `long` 8 adresinde saklanır;

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    long* ptr = (long*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr)); // 0
    ptr++; // bellekteki bir sonraki long adresini alır
    Console.WriteLine(new IntPtr(ptr)); // 8
    ptr+=4; // bellekteki 4 sonraki long adresini alır
    Console.WriteLine(new IntPtr(ptr)); // 40
}
```

Hiçbir değeri olmayan veri tipi anlamındaki `void` türü özeldir ve `void` göstericileri de özeldir. Veri tipi bilinmediğinde veya önemli olmadığına her şeyi ifade eden `void` göstericileri kullanılırlar. C ve C++ dilinin aksine boyuttan bağımsız yapıları nedeniyle, `void` göstericileri artırılmaz veya azaltılamaz:

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    void* ptr = (void*)IntPtr.Zero;
    Console.WriteLine(new IntPtr(ptr));
    ptr++; // Hata: The operation in question is undefined on void pointers
    Console.WriteLine(new IntPtr(ptr));
    ptr++; // Hata: The operation in question is undefined on void pointers
    Console.WriteLine(new IntPtr(ptr));
}
```

Herhangi bir gösterici tipi, **örtük** (**implicit**) bir dönüşüm kullanılarak `void*` tipine atanabilir;

```
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipne atama.
```

Bunun tersi durum ancak **açık** (**explicit**) olarak mümkündür;

```
int* p1 = (int*)IntPtr.Zero;
void* ptr = p1; //örtük olarak void* tipne atama.
```

```
int* p2 = (int*)ptr; //void* tip int* tipine örtük olarak atanamaz!
```

Referanstan Çıkarma İşleci Veri Tipinin Bir Parçasıdır

C ve C++ dillerinde, bir gösterici değışkeninin bildirimindeki yıldız işareti, bildirilen ifadenin bir parçasıdır. C# dilindeki bildirimdeki yıldız işareti **referanstan çıkarma işleci** (de-reference operator) olup veri tipinin bir parçasıdır;

Aşağıdaki örnekte verilen kod C# dilinde iki **int** göstericisi bildirmesine rağmen C ve C++ dillerinde ilki gösterici, ikincisi değışken olarak bildirilir.

```
int* a,b;
/*
 C ve C++ olarak eşdeğeri: int* a; int b;
 C# eşdeğeri: int* a; int* b;
*/
int *bir, *iki; //Hata:C# dilinde derlenmez.
/*
 Ancak C ve C++ dillerinde geçerli iki farklı göstericidir.
*/
```

Gösterici Üye Erişim İşleci

C ve C++ gibi C# dilinde de bir yapı ya da sınıf örneğinin üyelerine bir gösterici aracılığıyla erişmenin bir yolu olarak **->** işlecini kullanılır.

```
// See https://aka.ms/new-console-template for more information
unsafe
{
    Koordinat v;
    v.X = 5;
    v.Y = 10;
    Koordinat* ptr = &v;
    int x = ptr->X;
    int y = ptr->Y;
    string s = ptr->ToString();
    Console.WriteLine(x); // 5
    Console.WriteLine(y); // 10
    Console.WriteLine(s); // Koordinat
}

struct Koordinat
{
    public int X;
    public int Y;
}
```